

Word Embeddings

TF-IDF · Word2Vec · GloVe · Pre-trained Vectors

Mosharof Hossain

AI Engineer · Brain Station 23

Sources: Stanford NLP · Google Word2Vec · Gensim · Scikit-learn · HuggingFace

TF-IDF

Word2Vec

CBOW / Skip-gram

GloVe

Pre-trained

Why Do We Need Word Embeddings?

The journey from raw text to meaningful numbers

Machines can't read words. We need to convert text to numbers. But naive methods like one-hot encoding lose meaning. **Word embeddings place words in a vector space where similar words are geometrically close.**

The Evolution of Text Representation:

1

One-Hot

```
cat = [1,0,0,0]
dog = [0,1,0,0]
king = [0,0,1,0]
```

Size = vocabulary.
No similarity.

2

Bag of Words

```
'cat sat' = [1,0,1,0]
'dog sat' = [0,1,1,0]
```

Loses word order.
Sparse and high-dimensional.

3

TF-IDF

```
'cat' tf-idf=0.42
'the' tf-idf=0.01
rare = important
```

Captures word importance.
Still no semantics.

4

Word Embeddings

```
cat = [0.2,-0.4,0.8...]
dog = [0.3,-0.3,0.7...]
king = [0.5,0.8,-0.2...]
```

Dense vectors.
Similar words → similar vectors.

TF-IDF — Term Frequency · Inverse Document Frequency

Measures how important a word is in a document relative to a corpus

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

TF = frequency of term in document | IDF = $\log(N / df)$ | N = total documents | df = documents containing term

Worked Example — 3-Document Corpus:

D1: The cat sat on the mat

D2: The dog sat on the floor

D3: The cat chased the dog

Word	TF (D1)	DF	IDF	Score	Meaning
cat	1/6	2/3	0.405	0.068	Important
sat	1/6	2/3	0.405	0.068	Moderate
dog	0/6	2/3	0.405	0	Moderate
the	2/6	3/3	0	0	Stopword
mat	1/6	1/3	1.099	0.183	Most unique

TF-IDF is sparse and does not understand word relationships.

scikit-learn TF-IDF:

```
from sklearn.feature_extraction.text import
TfidfVectorizer

corpus = [
    "The cat sat on the mat",
    "The dog sat on the floor",
    "The cat chased the dog"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

# Sparse matrix → dense values
print(X.toarray())

features = vectorizer.get_feature_names_out()
print(features)
```

Word2Vec — Neural Word Embeddings

Google 2013 · Mikolov et al. · Words are known by the company they keep

Word2Vec trains a shallow neural network on text to produce dense vectors where **semantically similar words have similar vectors**.

Two Architectures:

CBOW — Continuous Bag of Words

Context → predict target word



→ **cat**

Average context vectors → predict centre word

Fast to train · Good for frequent words

Skip-gram

Target → predict context words



Given centre word → predict surrounding context

Better for rare words · Better on smaller datasets

Word2Vec Properties — Semantic Arithmetic

Meaning can be encoded geometrically in vector space

Vector Arithmetic:

king

- man

+ woman

≈ queen

More Word2Vec Analogies (A - B + C ≈ D):

Paris - France + Germany ≈ Berlin

doctor - hospital + school ≈ teacher

walked - walking + swimming ≈ swam

big - bigger + smaller ≈ small

```
model.wv.most_similar(positive=['king','woman'], negative=['man'])
```

Vector Space — Similar Words Cluster:

king

queen

man

woman

cat

dog

Paris

France

GloVe — Global Vectors for Word Representation

Stanford 2014 · Global word co-occurrence statistics

GloVe builds a global word co-occurrence matrix across the entire corpus and learns embeddings from these statistics. **It combines local context with global corpus information.**

Step 1 — Build Co-occurrence Matrix:

Corpus: 'the cat sat · the cat ate · the dog sat'

	the	cat	dog	sat	ate
the	-	2	1	2	1
cat	2	-	-	1	1
dog	1	-	-	1	-
sat	2	1	1	-	-
ate	1	1	-	-	-

Higher values = words appear together more often

Training Word2Vec with Gensim

From raw sentences to semantic vectors

Gensim is a popular Python library for Word2Vec, Doc2Vec, FastText, and loading pre-trained vectors.

Training your own Word2Vec model:

```
from gensim.models import Word2Vec

sentences = [
    ["the", "king", "ruled", "the", "kingdom"],
    ["the", "queen", "ruled", "wisely"],
    ["the", "man", "worked", "hard"],
    ["the", "woman", "studied", "science"]
]

model = Word2Vec(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=1,
    sg=0,
    epochs=10
)

model.save("word2vec.model")
```

Querying the model:

```
# Get vector
vec = model.wv['king']

# Similar words
model.wv.most_similar('king')

# Word arithmetic
model.wv.most_similar(
    positive=['king', 'woman'],
    negative=['man'])

# Cosine similarity
model.wv.similarity('cat', 'dog')

# Find odd word
model.wv.doesnt_match(
    ['cat', 'dog', 'car', 'wolf'])
```

Using Pre-Trained Embeddings

Google News · Stanford GloVe · HuggingFace Hub

Pre-trained embeddings are vectors already trained on massive corpora. Instead of training from scratch, **load them and use them immediately.**

Google Word2Vec

Google News

300-dimensional vectors
100B-word training corpus
3M-word vocabulary

```
from gensim.models import
    KeyedVectors

wv = KeyedVectors.load_word2vec_format(
    "GoogleNews-vectors-300.bin.gz",
    binary=True)
```

GloVe

Stanford NLP

50 / 100 / 200 / 300 dims
Wikipedia + Gigaword
CommonCrawl variants

```
import gensim.downloader

wv = gensim.downloader.load(
    "glove-wiki-gigaword-100"
)
```

HuggingFace Hub

Modern Transformer Models

Thousands of models
BERT / RoBERTa
Sentence Transformers

```
from transformers import
    AutoTokenizer, AutoModel

tok = AutoTokenizer.from_pretrained(
    "sentence-transformers/
    all-MiniLM-L6-v2")
```

Common operations:

```
wv.most_similar('python')
wv.doesnt_match(['breakfast', 'lunch', 'dinner', 'car'])
```

```
wv.similarity('cat', 'kitten')
wv['king'] - wv['man'] + wv['woman']
```


TF-IDF vs Word2Vec vs GloVe — Full Comparison

Choosing the right representation for your task

Feature	TF-IDF	Word2Vec	GloVe
Type	Sparse statistical	Dense neural	Dense global
Vector size	Vocabulary size	100–300d	50–300d
Semantics	No	Yes	Yes
Word relationships	No	Strong	Strong
Training	Seconds	Minutes–hours	Minutes–hours
Best for	IR / classification	Custom domain	Pre-trained vectors
Main strength	Word importance	Semantic context	Global statistics
Main weakness	No semantics	Needs training data	Large matrix
Library	Scikit-learn	Gensim	Gensim

Demo Plan + Session Summary

Run the concepts live and see embeddings working

Live Demo Steps

1

TF-IDF on 3 Documents

Sparse matrix and important words

2

Train Word2Vec

Custom corpus → 100d vectors

3

CBOW vs Skip-gram

Compare sg=0 and sg=1

4

Load GloVe

Use gensim downloader

5

Pre-trained Queries

Similarity and analogy

6

Visualise with PCA

100d → 2d scatter plot

7

IMDb Classification

TF-IDF vs Word2Vec

Session Summary

TF-IDF

Sparse; captures word importance.

Word2Vec

Dense; learns local semantic context.

GloVe

Dense; uses global co-occurrence.

Pre-trained

Ready-to-use vectors from huge corpora.

Vector arithmetic

Meaning can be encoded geometrically.

Choosing wisely

Match representation to your task.